

# Derivations

## Concepts of Programming Languages Lecture 6

# Practice Problem

$$(a+b)^2 = a^2 + b^2 + 2ab$$

Recall the **=** operator in OCaml. Write down (to the best of your ability) the syntax, typing, and semantic rules for **=**

$$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle = \langle \text{expr} \rangle$$

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 = e_2 : \text{bool}} \quad (\text{eq})$$

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 = v_2}{e_1 = e_2 \Downarrow \text{true}} \quad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \neq v_2}{e_1 = e_2 \Downarrow \text{false}}$$

# Practice Problem

$$\frac{\{\} \vdash 2 : \text{int} \quad \{\} \vdash x : \text{int}}{\{\} \vdash \text{let } x = 2 \text{ in } x + x : \text{int}}$$

Some students suggested this rule in the previous class for let-expressions. What is the problem in this rule?

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash x : \tau_1 \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \text{(let)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau}$$

# Outline

- » Discuss derivations in general
- » See how to read and write derivations
- » Go through a couple examples

$$\frac{\mathcal{P} \vdash e_1 : \text{int} \quad \mathcal{P} \vdash e_2 : \text{int}}{\mathcal{P} \vdash e_1 + e_2 : \text{int}}$$

# Recap

# Let-Expressions

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \text{ (let)}$$

$$\frac{e_1 \Downarrow v_1 \quad [v_1/x]e_2 \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v} \text{ (letEval)}$$

# If-Expressions

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{(if)}$$

$$\frac{e_1 \Downarrow \text{true} \quad e_2 \Downarrow v_2}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_2} \text{(ifEvalTrue)}$$

$$\frac{e_1 \Downarrow \text{false} \quad e_3 \Downarrow v_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_3} \text{(ifEvalFalse)}$$

# Derivations

# High Level

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau}$$

|                                                                                       |                                                                          |                                                                   |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|
| $\frac{}{\{\} \vdash 2 : \text{int}} \text{ (intLit)}$                                | $\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{ (var)}$        | $\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{ (var)}$ |
|                                                                                       | $\frac{}{\{y : \text{int}\} \vdash y + y : \text{int}} \text{ (intAdd)}$ |                                                                   |
| $\frac{}{\{\} \vdash \text{let } y = 2 \text{ in } y + y : \text{int}} \text{ (let)}$ |                                                                          |                                                                   |

Derivations *prove* that a judgment holds w.r.t some rules

A **derivation** is a tree in which:

- » each node is labeled with a judgment
- » and judgment *follows* from the judgments at its children by an **inference rule**

# Applying Rules

$$\frac{}{\Gamma \vdash [] : \tau \text{ list}} \text{ (nil)} \quad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \text{ list}}{\Gamma \vdash e_1 :: e_2 : \tau \text{ list}} \text{ (cons)}$$

$$(5+4)^2 = 5^2 + 4^2 + 2 \cdot 5 \cdot 4 \quad (a+b)^2 = a^2 + b^2 + 2ab$$

|                                                             |                                                  |
|-------------------------------------------------------------|--------------------------------------------------|
| $\{x : \text{int}\} \vdash x + 1 : \text{int}$              | $\{x : \text{int}\} \vdash [] : \text{int list}$ |
| $\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}$ |                                                  |

So far, we've used rules as ways of describing the behavior of a PL

When we build typing derivations, we *instantiate* the meta-variables in the rule at *particular* expressions, contexts, etc.

# Building from the Ground Up

|                                                                                                                                                                                      |                                                                         |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--|
| $\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$                                                                                                                    | $\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$    |  |
| $\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$                                                                                                             | $\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$ |  |
| $\frac{\{x : \text{int}\} \vdash x + 1 : \text{int} \quad \{x : \text{int}\} \vdash [] : \text{int list}}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$ |                                                                         |  |

But we can't *just* apply rules, because it's possible that the premises of a rule **also need to be demonstrated**

This is how we get our tree structure: we apply rules from the ground up

# Axioms (When are we done?)

|                                                                          |                                                                                     |  |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--|
| $\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$        | $\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$                |  |
| $\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$ | $\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$             |  |
|                                                                          | $\frac{}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$ |  |

The leaves of the tree are **axioms**, i.e., a rules with no premises

In our case, this will almost always be "literal" or "variable" rules

# Integer Literals

(1)

$$\frac{n \text{ is an int lit}}{\Gamma \vdash n : \text{int}} \quad (\text{intLit})$$

(2)

$$\frac{n \text{ is an int lit}}{n \Downarrow n} \quad (\text{intLitEval})$$

1. If  $n$  is an integer literal, then it is of type  $\text{int}$  in any context
2. If  $n$  is an integer literal, then it evaluates to the number it represents

# A Note about Side Conditions

we don't write "1 is an integer literal"

|                                                                                     |                                                                         |  |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--|
| $\frac{}{\{x : \text{int}\} \vdash x : \text{int}} \text{ (var)}$                   | $\frac{}{\{x : \text{int}\} \vdash 1 : \text{int}} \text{ (intLit)}$    |  |
| $\frac{}{\{x : \text{int}\} \vdash x + 1 : \text{int}} \text{ (intAdd)}$            | $\frac{}{\{x : \text{int}\} \vdash [] : \text{int list}} \text{ (nil)}$ |  |
| $\frac{}{\{x : \text{int}\} \vdash (x + 1) :: [] : \text{int list}} \text{ (cons)}$ |                                                                         |  |

If a premise is a side-condition this *it is not included in the derivation*

Side conditions need to hold in order to apply the rule, but they don't appear in the derivation itself

We will always make side conditions clear

# Float Literals

$$(1) \quad \frac{n \text{ is an float lit}}{\Gamma \vdash n : \text{float}} \quad (\text{floatLit})$$

$$(2) \quad \frac{n \text{ is an float lit}}{n \Downarrow n} \quad (\text{floatLitEval})$$

1. If  $n$  is an float literal, then it is of type float in any context
2. If  $n$  is an float literal, then it evaluates to the number it represents

# Boolean Literals

$$(1) \quad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{ (trueLit)}$$

$$(2) \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{ (falseLit)}$$

$$(3) \quad \frac{}{\text{true} \Downarrow \text{true}} \text{ (trueLitEval)}$$

$$(4) \quad \frac{}{\text{false} \Downarrow \text{false}} \text{ (falseLitEval)}$$

1. `true` is of type `bool` in any context
2. `false` is of type `bool` in any context
3. `true` evaluates to the value  $\top$
4. `false` evaluates to the value  $\perp$

# Variables

$$\frac{(\cancel{x} : \tau) \in \Gamma}{\Gamma \vdash \cancel{x} : \tau} \quad \text{var} \quad \cancel{(\text{intLit})}$$

If  $v$  is declared to be of type  $\tau$  in the context  $\Gamma$ , then  $v$  is of type  $\tau$  in  $\Gamma$

**Variables cannot be evaluated** (more on this when we talk about substitution and well-scopedness)

# Back to the Example

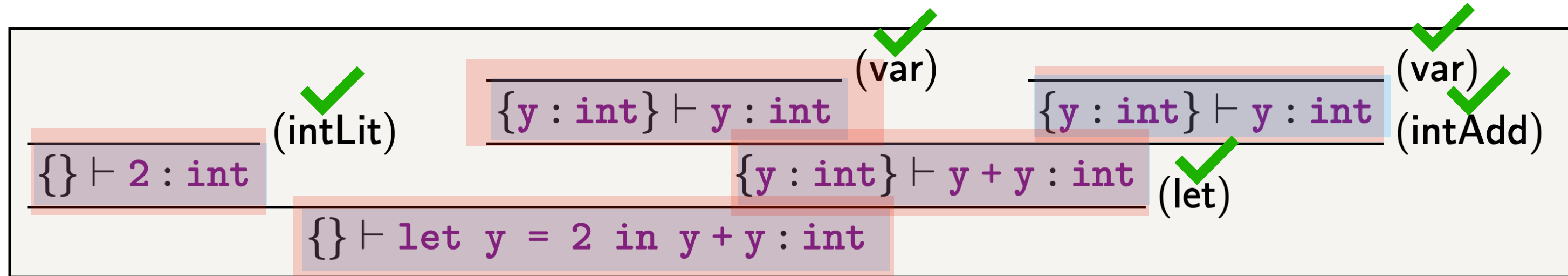
$$\frac{\frac{}{\{\} \vdash 2 : \text{int}} \text{(intLit)} \quad \frac{\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{(var)} \quad \frac{\frac{}{\{y : \text{int}\} \vdash y : \text{int}} \text{(var)}}{\{y : \text{int}\} \vdash y + y : \text{int}} \text{(intAdd)}}{\{\} \vdash \text{let } y = 2 \text{ in } y + y : \text{int}} \text{(let)}$$

We need  $\{\} \vdash 2 : \text{int}$  in order to proof that the bottom typing judgment holds

Now we know that this follows from the **intLit** rule, which says that 2 is always an int, *by fiat*

Okay, I know that was a  
lot, let's take a step back

# Derivations Encode Natural Language Arguments



$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (let)}$$

$$\frac{\text{n is an integer literal}}{\Gamma \vdash \text{n} : \text{int}} \text{ (intLit)}$$

The expression `let y = 2 in y + y` is an `int` because

» `2` is an `int` by fiat (and so `y` is being assigned to a well-typed

» and, assuming `y` is an `int`, `y + y` is an `int` because

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ (addInt)}$$

- `y` is an `int` (by assumption)
- and so is `y` (by assumption)

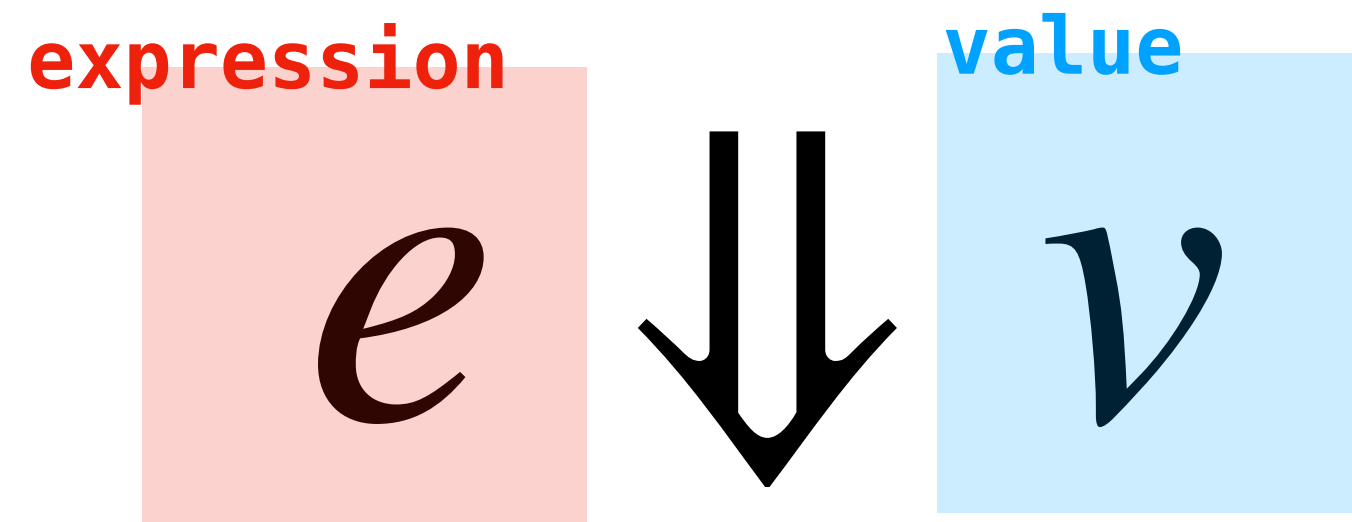
$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \text{ (var)}$$

and so integer-adding these two expressions (`y` and `y`) yields an `int`

and so assigning `y` to `2` in `y + y` yields an `int`

And all this works for  
semantics judgements as well

# Recall: Semantic Judgements



A **semantic judgment** is a compact way of representing the statement:

*The expression  $e$  evaluates to the value  $v$*

A **semantic rule** is an inference rule with semantic judgments

# Recall: Integer Addition Semantic Rule

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 + v_2 = v}{e_1 + e_2 \Downarrow v} \text{ (evalInt)}$$

*If  $e_1$  evaluates to the (integer)  $v_1$  and  $e_2$  evaluates to the (integer)  $v_2$ , and  $v_1 + v_2 = v$ , then  $e_1 + e_2$  evaluates to the (integer)  $v$*

# Semantic Derivations

$$\frac{\frac{}{\text{true} \Downarrow \text{true}} \text{ (trueEval)} \quad \frac{}{2 \Downarrow 2} \text{ (intEval)}}{\text{if true then 2 else 3} \Downarrow 2 \text{ (ifEval)}}$$

We can also write derivations to prove semantic judgments

The principle is the same, except that the judgments are semantic judgments instead of typing judgments

# Examples

# Example (Typing)

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}$$

$$\frac{\begin{array}{ccc} \frac{}{\{\} \vdash \text{true} : \text{bool}} \text{ (trueLit)} & \frac{}{\{\} \vdash 2 : \text{int}} \text{ (intLit)} & \frac{}{\{\} \vdash 5 : \text{int}} \text{ (intLit)} \end{array}}{\{\} \vdash \text{if true then 2 else 5} : \text{int}} \text{ (if)}$$

# Example (Evaluation)

if true then 2 else 5  $\Downarrow$  2

# Example (Typing)

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 <? e_2 : \text{bool}}$$

$$\frac{\frac{\{\} \vdash 2 : \text{int} \quad \{\} \vdash 3 : \text{int}}{\{\} \vdash 2 + 3 : \text{int}} \text{ (addInt)}}{\{\} \vdash 2 + 3 : \text{int}} \text{ (intLit)}$$

$$\frac{\{\} \vdash 4 : \text{int}}{\{\} \vdash 2 + 3 <? 4 : \text{bool}} \text{ (intLit)}$$

$$\{\} \vdash 2 + 3 <? 4 : \text{bool}$$

# Example (Evaluation)

$2 + 3 \lessgtr 4 \Downarrow \text{true}$

# Example (Evaluation)

$$\frac{e_1 \Downarrow v_1 \quad [v_1/x]e_2 \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v}$$

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 + v_2}{e_1 + e_2 \Downarrow v}$$

$$\frac{\frac{2 \Downarrow 2 \quad 2 \Downarrow 2}{2 + 2 \Downarrow 4}}{2 \Downarrow 2}$$

$$\text{let } x = 2 \text{ in } x + x \Downarrow 4$$

# Summary

Derivations are **tree-like proofs** that judgments hold with respect to a collection of inference rules

Derivations are **compact mathematical representations** of English language arguments

Learning to write derivations takes *practice*